Kotlin from Scratch

Part I: Programming Fundamentals

Chapter 1 - Kotlin Basics

- Comments include `// one line comments`, `/* multi-line comments */`, and `/** documentation comments */`.
- Naming Conventions
  - Package names are lowercase letters with no underscores. Join multiple words or use camelCase. Use reverse domain notation. Ex: `org.example.myProject`. Avoid camelCase package names in Android projects which have stricter conventions. Just use one word or multiple words pushed together all lowercase. For personal use consider `io.github.llcawthorne.mypackage`.
  - Class names are PascalCase. Choose words that are nouns or noun phrases. Ex: `FlightSimulation`.
  - Function and method names are camelCase. Use verbs or verb phrases. Ex: `calculateShortestPath()`
  - Variable names are single words or camelCase for multiple words. Choose a word that describes the purpose, function, or property of the variable. Ex: `username`
  - Constants or final variables are all uppercase with underscores to separate words. Ex: `MAX_VALUE`
- Unadorned numbers are `Int` or `Double` if they have a decimal. Longs are `10L`, and Floats are `3.14f`.
- The default setting on most IDE's is to disable implicit casting. Use a type-casting method to convert such as `toByte()`, `toInt()`, `toLong()`, `toShort()`, `toDouble()`, `toFloat()`, `toChar()`, or `toString()`.
- `"Hello world!".toInt()` throws a `NumberFormatException`. `"42".toInt()` is fine.
- Arithmetic operators are `+`, `-`, `*`, `/`, and `%`.
- When you divide to `Int` the result is `Int`. The fraction is discarded. So call `toDouble()` or `toFloat()` on at least one for a decimal.

- Assignment operators are `=`, `+=`, `-=`, `*=`, and `/=`.
- `+` and `+=` work with `String`s, as does `==` and `>` / `<`. Strings are compared lexicographically, character by character, based on their Unicode values.
- `++` and `--` are the Unary operators.
- Relational operators are `==`, `!=`, `>`, `<`, `>=`, and `<=`.
- Logical operators are `&&`, `||`, and `!`.
- `msg[1]` is the second character of the `String` `msg`. `'e'` for `"Message"`.
- You can build strings with `+`, `.plus`, or the `buildString` function:

```
val c = buildString {
    append("Hello,")
    append(" ")
    append("world!")
}
println(c) // output: Hello, world!
```

- *String templates* use `$` to signal that what follows should be treated as code instead of literal text. More complex expressions require `{}` after the dollar sign. `My name is $name` or `$x + $y = ${x + y}`.
- *Escape sequences* represent character hard to input directly. They include `\n`, `\t`, `\$`, and `\\`.
- You cannot hold `null` in normal variables. A *nullable type* is declared with a `?` at the end of the type like `String?`.
- The Kotlin *safe call operator* (`?.`) allows you to call a method on a nullable objects and returns `null` if the object is `null`.
- The *Elvis operator* (`?:`) is used with the *safe call operator* to return a default value other than `null`. Ex: `val len = str?.length ?: -1`
- The *null assertion operator* or *double-bang operator* (`!!`) asserts that a value is not `null` and throws `NullPointerException` if it is `null`. Ex: `val length = name!!.length`
- The two primary conditional statements are `if` and `when`.

```
val a = 100
val b = -30
val max: Int    // a val can be initialized later but do so before using

if (a > b) {
    max = a
    println("a is greater than b.")
    println("max of $a and $b is: $max")
} else if (a < b) {
    max = b
    println("b is greater than a.")
    println("max of $a and $b is: $max")
} else
    println("a and b have the same value: $a")
```

- `when` checks a value against multiple conditions and executes the code block of the first matching condition. It can also have an `else` clause for when the value doesn't match any condition.

```
val x = 5

when {
    x > 0 -> println("x is positive")
    x == 0 -> println("x is zero")
    x < 0 -> println("x is negative")
    else -> println("x is not a real number")
}
```

- You can also provide the variable after the `when` in parentheses. It compares for equality or has special forms for `in` and `is` tests.

```
val hour = 13

when (hour) {
    in 0..11 -> println("Good morning")
    in 12..16 -> println("Good afternoon")
    in 17..23 -> println("Good evening")
    else -> println("Invalid hour")
}
```

- Notice the lack of a `break` statement in the `when` code blocks.

- Loops allow you to repeat a block of code. Kotlin includes `for` loops, `while` loops, and `do...while` loops.
- `for (i in 1..4)` loops four times. `for (i in 1 until 10 step 3)` loops three. And `for (i in 4 downTo 1)` loops four.
- `continue` and `break` work as expected in all loops.
- You can label a loop and use a labelled `break` or `continue`:

```
loop1@ for (i in 1..5) {
    loop2@ for (j in 1..5) {
        print("$i,$j")
        if (i == j) continue@loop1  // print up to the diagonal
    }
    println()
}
```

- `sqrt` and `pow` are in `kotlin.math` and must be `import`'ed.
- *function parameters* are treated as implicitly read-only within the function.

```
fun functionName(parameter1: Type,
                 parameter2: Type, ...): ReturnType {
    // function body
}
```

- Kotlin can infer the return type of a *single-expression function*. Ex: `fun add(x: Int, y: Int) = (x + y)`
- You can declare default values for parameters. Ex: `fun greet(name: String, greeting: String = "Hello") {`
- You can use *named arguments* in any order when calling a function. Ex: `greet(greeting = "Good morning", name = "Bob")`
- Kotlin supports *function overloading* with different numbers or types of parameters.
- You can refer to a function without calling it with `::greet`. Ex: `val greetingFunction = if (useGreeting) ::greeting else ::farewell`

- `run`, `with`, `let`, `also`, and `apply` are the *scope functions*.

```
val result = run {
    val x = 10
    val y = 20
    x + y // the value of this final expression is returned.
}
println("Result: $result") // prints "Result: 30"
```

- *Lambda expressions* let you define and pass around blocks of function-like code.
  Ex: `val greet : (String) -> String = {name -> "Hello, $name"}`
  Or: `val greet = {name: String -> "Hello, $name"}`
  Or even: `val greet = {name -> "Hello, $name"}`
- If a lambda body is a single expression, that expression will be returned.
- If a lambda has a single parameter, you can omit its declaration and use the implicit `it` keyword to stand-in for the parameter. Ex:
  Instead of `val square: (Int) -> Int = { a -> a * a }` you can use `val square: (Int) -> Int = {it * it}`
- `readln()` gets input until ENTER. All input is treated as text.

```
while(true) {
    print("Enter an integer: ")
    val num = readln()

    // Validate using a try...catch block.
    try {
        val intValue = num.toInt()
        println("You entered: $intValue")
        break   // Stop the loop on success
    } catch (e: NumberFormatException) {
        println("Invalid input. Try again.")
    }
}
```

- Kotlin uses Java functionality for file IO:

```
import java.io.File
import java.util.Scanner

fun main() {
    // Replace the path below with the path to your file.
    val inputFile = "inputfile.txt"

    try {
        val file = File(inputFile)
        val sc = Scanner(file)
        while (sc.hasNextLine()) {
            val line = sc.nextLine()
            println(line)
        }
    } catch (e: Exception) {
        println("An error occurred: ${e.message}")
    }
}
```

- `File` objects have an `appendText()` method that takes a `String`.

```
import java.io.File

fun main() {
    // Replace the file locations as needed.
    val inputFile = File("inputfile.txt")
    val outputFile = File("outputfile.txt")

    // Read all lines from the input file.
    val lines = inputFile.readLines()

    // Write all lines to the output file.
    for (line in lines) {
        outputFile.appendText("$line\n")
    }
    println("Copied inputfile.txt to outputfile.txt")
}
```

Chapter 2 - Arrays, Collections, and Classes

- A Kotlin *array* is a collection of elements in a contiguous block of memory. The size of the array is determined when the array is created. You create arrays with `arrayOf`. An

`Array<Any>` can hold any non-nullable types. You can also create arrays with `val num = Array(4) { i -> i * 2 }` for `[0, 2, 4, 6]`. The 4 is the size and the function determines elements based on index.

- Primitive arrays are good for perfomance critical operations and created via `intArrayOf`, `doubleArrayOf`, etc. There is no primitive for `String`.
- A two dimensional 3x4 array is `val twoDimArray = Array(3) { Array(4) {0} }` or through nested calls to `arrayOf`.
- A list supports `size`, `contains`, `indexOf`, `subList`, `first`, and `last`.
- A mutable list also supports `add`, `remove`, `removeAt`, `removeAll`, `clear`, `+=` to add an element, and `-=` to remove from the head.
- A set is a collection of unique elements. Sets don't have order.
- Sets also have list methods and `union`, `intersect`, and `subtract` to return new sets from the result of operating on two sets.

```kotlin
fun <T> powerSet(set: Set<T>): Set<Set<T>> {
    var subsetsSoFar: MutableSet<Set<T>> = mutableSetOf(emptySet())

    for (element in set) {
        val newSubsets: MutableSet<Set<T>> = mutableSetOf()

        for (existing in subsetsSoFar) {
            val combined: MutableSet<T> = mutableSetOf()
            combined.addAll(existing)
            combined.add(element)
            newSubsets.add(combined)
        }

        subsetsSoFar.addAll(newSubsets)
    }

    return subsetsSoFar
}

// the same thing as a one-liner with fold
fun <T> powerSetFold(set: Set<T>): Set<Set<T>> {
    return set.fold(setOf(emptySet())) { subsetsSoFar, element ->
        subsetsSoFar + subsetsSoFar.map { it + element }
    }
}
```

- A *map* is a collection of key-value pairs. You commonly use `size` property and `get`, `put`, `remove`, and `containsKey` methods. Note that `put` takes two arguments, key and value, and not "key to value" like `mapOf`.
- These collections are created with `listOf`, `setOf`, `mapOf` and the corresponding `mutableListOf`, `mutableSetOf`, and `mutableMapOf` functions.
- A *class* in Kotlin is a template for creating custom objects. It defines classes and methods that all objects for all objects of that class.
- The *primary constructor* is defined in the class header, a set of parentheses after the class name. It sets properties of new objects.
- A class can also have *secondary constructors* defined in the class body with the `constructor` keyword. If the class has a primary constructor, the secondary constructors must always call it using the `this` keyword.

```
class Car(val make: String, val model: String, val year: Int) {
    // property initialization inside class body
    var color: String = "Unknown"

    // no arg secondary constructor
    constructor() : this("Unknown", "Unknown", 0)

    constructor(make: String) : this(make, "Unknown", 0)

    constructor(make: String, model: String) : this(make, model, 0)

    override fun toString(): String =
        "Make: ${make}, Model: ${model}, Year: ${year}, Color: ${color}"
}
```

- The above example could have been done with default values.
- You can provide an `init` block within a class to run code during object construction. Multiple `init` blocks will run in order of declaration.
- All properties and methods are `public` unless declared otherwise. It's still possible to set `private` properties through the constructor, but further access can be mediated by methods. `private` is only accessible within the class, `protected` in the class and subclasses, and `internal` only within the same module.
- A module is a set of Kotlin files processed together during compilation. Anything accessed through `import` is not part of the module.
- `this` refers to the current instance of the class and is useful to distinguish between properties and parameters with the same name.
- Kotlin classes aren't inheritable by default and need to be marked `open`. Both the class itself and individual methods to be overridden need to be `open`. The child class uses `override` for the properties or methods.
- A **data class** is a simple class used to hold data. It often doesn't have user-defined methods. It overrides `equals`, `toString`, and `hashCode` plus it possesses a `copy` method to produce shallow copies.
- `Pair` and `Triple` hold two or three values of same or different types.
- An **abstract class** can't be instantiated and is defined for inheritance.

- An **interface** is a collection of functions and properties that must be implemented by inheriting classes or types.
- An **enum class** represents a group of constants with optional properties and methods and is used for a fixed set of values. They define a common set of behaviors for classes that implement the interface.
- Data classes support *deconstruction*:

```
data class Person(val name: String, val age: Int)
val person = Person("Steve", 40)
val (the_name, age) = person // variables name same or different; location matches
```

- The infix `to` method just generates a `Pair`. `Pair` and `Triple` values are accessible via `pair.first` or `triple.third` and they deconstruct also.
- `copy` makes a shallow copy. To make a deep copy, you need a custom function.
- You can make a deep copy of a list by mapping members `copy` operation

```
val originalList = mutableListOf(Person("Alice", 30), Person("Bob", 25))
val deepCopyList = originalList.map { it.copy() }.toMutableList()
```

Chapter 3 - Visualizing with JavaFX

- Unsurprisingly, this chapter was about JavaFX. Look at the projects' code to see how it works. The chapter does have a pretty good description of the different UI components, so re-read it if you want to get jiggy with JavaFX.

Part II - Applications in Math and Science

Chapter 4 - Solving Mathematical Problems with Code

So, I typed all the code examples in this chapter. The author has a bad habit of using mutable var's and also doing it with global variables, so overall I don't consider it the good, clean Kotlin code we were promised. Also, he left a function out of Project 16, so that example was uncompilable. I typed it all up and deleted it when it didn't work, but then I looked on github and it was both mentioned in ERRATA and the function was defined in the example code, so I lifted

the whole Random Walk example from Github. Honestly, I don't feel like typing all this JavaFX boilerplate for the Modeling and Simulation and Fractals chapters, so I may skip those chapters or else lift them from Github and skim the description. I'm still looking forward to Graph Algorithms, and the author seems to tone down `var` use a lot in those chapters, so I'm not completely done with this book.

Chapter 5 - Modeling and Simulation

Part III: Recursion, Sorting, and Searching

Chapter 6 - Recursive Functions and Fractals

Chapter 7 - Sorting and Searching

These examples were all pretty short except A*, so I typed them all in by hand again to get practice typing the language. I still had a bad habit of using `def` where I needed `fun`.

Part IV: Optimization with Nature-Inspired Algorithms

Chapter 8 - The Genetic Algorithm

Chapter 8 has a detailed several page discussion of the logic of genetic algorithms, along with a general discussion of optimization problems, but I feel like the genetic algorithm code of the three projects we did demonstrates the process quite clearly.

Chapter 9 - Agent-Based Algorithms

We used Particle Swarm Optimization to solve the Eggholder problem, and it actually zoned in on the best solution more often than the genetic algorithm while doing it in half as many generations. It also found the second best solution the genetic algorithm found plus another third best solution that I never found in multiple runs of the genetic algorithm; plus it didn't get bogged down by the same "not very good" solution that the genetic algorithm kept finding. We used the ant simulation to solve the traveling salesman problem for a 52 node graph with distances based on Berlin.

It's hard to understand particle swarm optimization algorithms without the equations and graphs of the book. The basic idea is each particle has a certain amount of inertia that causes its velocity to carry it forward, plus it steers itself towards the best position it personally has

found, and it steers itself towards the best location found by the swarm. There are three weights that are set to constant values for the problem that affect each of these tendencies. You can call these three values the *inertia factor*, the *particle memory/cognitive factor*, and the *swarm memory/social factor*. There are also two random values between 0 and 1 that are used in the calculations. Note that particle swarm optimization is the first problem difficult enough that the author felt it necessary to provide pseudocode explanations in addition to code samples. Both of these optimization problems were fairly significant blocks of code and I was looking forward to my next book, so I took the easy way out and used the code from github instead of typing it all by hand. The only mistakes I made typing the genetic algorithm were being sloppy with parentheses when calculating the min of the max (I tried to pass max three parameters by mistake) and not updating one of my IntArray's to a DoubleArray while re-using code from a previous example. The examples weren't really any larger than the genetic algorithm, but I got used to not having to start from scratch since we did three of the same thing. I've completely gotten used to "Kotlin things" that made me want to type in some code, like `fun` over `def` and `println`.